

2.7 LIST OF LAB ASSIGNMENTS (SESSION WISE)

In this section List of Problems is given for deep dive into real programming. Unit wise problem/exercise given for your lab. You can use Apache NetBeans IDE or Eclipse IDE for doing the following Lab Exercises:

SESSION 1: BASIC OF SERVLET

1. Write a Servlet Programme to print the current date and time along with the timestamp.
2. Create an HTML form with the input of student information using HTTP Protocol and method, then display the input information using Servlet.
3. Write a servlet program to capture client IPs and display it.
4. Write a servlet program for session management using HTTP Session along with tracking and also use a cookie for session tracking.
5. Write a CRUD (Create/Save, Read, Edit/Update, Delete) application using servlet. Create a Database named IGNOU, create a table named Student which must capture the student information (basics, contact, enrollment details along with courses). Make necessary assumptions required.

SESSION 2: JSP

1. Write JSP Programme to print current date and time along with timestamp, implement auto-refresh of a page.
2. Create a JSP page and implement a Scripting Tag, Expression tag and Declaration tag.
3. Import JSTL library in JSP Page and use its following tags:
 - a. out
 - b. if
 - c. forEach
 - d. choice, when and otherwise
 - e. url and redirect
4. Create a JSP Page for database connectivity using JDBC and show the students details from the database created during exercise no 5 in session 1.
5. Write a JSP application using following Action Elements
 - a. jsp:forward
 - b. jsp:include
 - c. set & getProperty
 - d. jsp:useBean
6. Write a JSP program using the following implicit objects with an example:
 - a. Out
 - b. request
 - c. response
 - d. Session
 - e. pageContext
 - f. exception
7. Create a JSP Project implementing all the above (Session 1 and Session 2) concepts. Login Form, CRUD operation of Student details, Session Management with exception handling using Servlet and JSP. Make necessary assumptions required.

SESSION 3: MAVEN, FRAMEWORK FOR J2EE

1. Create a Maven-based project using Spring Initializr.
2. Create a Maven-based J2EE application for Spring MVC.

3. Create a Maven-based J2EE application for Hibernate and JPA. Use the same database created in Exercise no. 5 of Session 1.
4. Define a new implementation of Teacher Interface for his/her favourite Course in Spring using Inversion of Control and retrieving the information from the new teacher implementation.

SESSION 4: DEPENDENCY INJECTION, CONTROLLER

1. Write a class and implement it using dependency injection.
2. Write the service interface with the getRandom() method, define 3 courses in an array, and inject using dependency injection into the teacher Interface(exercise no 4 in session 3). Test the application and verify the retrieving of random courses.
3. Write a programme using Spring Framework to create a controller and display response in view using @GetMapping() annotation.
4. Create a Form to capture Student Admission information (make usual assumptions about the attributes) using Spring Form tags (must use Text, textarea, dropdown, date picker, true/false and checkbox) and write a controller using @PostMapping() to display the form information.

SESSION 5: FORM VALIDATION, BOOTSTRAP AND CSS

1. Apply the client validation in the form created in the above exercise 4 of session 4, along with server-side validation.
2. Write a programme to bind form objects with entity bean in Spring MVC.
3. Configure Bootstrap in Spring MVC and use default styling classes in the form and view created in the above exercises.
4. Apply custom Styling to your pages in Spring MVC.

SESSION 6: HIBERNATE, JPA

1. Create a database for the student admission lifecycle and configure Hibernate and JPA with the Spring MVC Project along with all table entities.
2. Retrieve Student information using Hibernate and printing the value in the console.
3. Create CRUD (Create/Save, Read/Fetch, Edit/Update, Delete) using Spring MVC and Hibernate.
4. Apply batch update for student's admission approval using Spring MVC and Hibernate.

SESSION 7: SPRING BOOT AND REST CONTROLLER

1. Create a Spring Boot application using Spring Initializer. Add the following dependencies manually:
 - a. Spring MVC
 - b. Hibernate
 - c. JPA
 - d. Thymeleaf
 - e. DevTool
 - f. Actuator
 - g. MySQL/MSSQL/Oracle/MongoDB (as per your choice) driver.
2. Configure Database settings through the property file in Spring Boot.
3. Create JPA Repositories for all entities used in the Student Admission lifecycle.

4. Create Rest Controller to fetch Student Information using JPA Repository; the response should display in JSON format.

SESSION 8: REST API, SPRING SECURITY, ACTUATOR

1. Add actuator dependency in Spring boot and Test Actuator (all available Endpoints).
2. Create REST API for CRUD operation using Spring Boot and Hibernate/JPA using annotation.
3. Create a REST API for CRUD operation using Spring Boot and Hibernate/JPA using XML configuration.
4. Add Spring Security dependency in the existing spring boot application created in Exercise 2 of Session 8 and configure it as the default login.

SESSION 9: SPRING SECURITY

1. Create a User table into the database and bind the user entity with Spring Security for Login.
2. Create a Custom Login Page in HTML and authenticate using Spring Security in Spring Boot.
3. Implement logout functionality in Spring Security.
4. Create a User registration form and validate the form. Once information is validated and saved, write functionality to auto-login using Spring Security.

SESSION 10: BOOTSTRAP, ROLE BASED AUTHENTICATION AND CSRF

1. Add Bootstrap styling in the Login and registration page in the above exercises in session 9.
2. Create a Role Table in the database and configure it with Spring Boot, Security and Hibernate.
3. Write code for role-based authentication using Spring Security.
4. Display current logged in User details on the dashboard along with client IP, data time and user's current role.
5. Add CSRF functionality in the authentication.
6. Restrict role-based access to views in Spring Boot Security.

2.8 REFERENCES

- Download JDK: <https://www.oracle.com/in/java/technologies/javase-downloads.html>
- JDK Installation Document for Multiple Platform: <https://docs.oracle.com/en/java/javase/16/install/overview-jdk-installation.html#GUID-8677A77F-231A-40F7-98B9-1FD0B48C346A>
- Java Environment PATH AND CLASSPATH Setup for the various platform: <https://docs.oracle.com/javase/tutorial/essential/environment/paths.html>
- Download NetBeans IDE: <https://netbeans.apache.org/download/index.html>
- Download Eclipse IDE: <https://www.eclipse.org/downloads/>
- Download Apache Tomcat Web Server: <https://tomcat.apache.org/download-10.cgi#10.0.8>